Weekly Python Assignment 7 - Group 3 - S23

In [45]:
```
# For printing basic infromation

def Sehyoun_Jang(prob):
    print("Name: Sehyoun Jang")
    print("sej324@lehigh.edu")
    print("Assignment ENGR10-S23-PY 7-3, Problem:", prob)
    print("GROUP 3")
    print("----------------------------------------------------------------")
```

1(M). Define a function called letter_count that accepts a string parameter. It should create a dictionary called letter_dict, with its keys being the unique letters of the inputted string parameter and the corresponding values being the number of instances a letter appears in the string. The function will print out letter_dict as is, and then print out each key and its corresponding value on a separate line, with a colon (:) separating the two. Lastly, the function will return letter_dict. Call letter_count function using the following string: "Four score and seven years ago our fathers brought forth on this continent, a new nation, conceived in Liberty, and dedicated to the proposition that all men are created equal."

Save the returned dictionary as char_count. Use Python to determine and print out the letter that appears the **third** most frequent, along with its number of instances.

Note:

- A capitalized letter should not be counted as a separate letter from its lowercase counterpart (i.e., "Aa" is counted as 2 instances of "a")
- A space or a symbol (e.g., comma, period,...) does not count as a letter.

In [46]:
```
Sehyoun_Jang(1)

def letter_count(sentence):
    letter = "abcdefghijklmnopqrstuvwxyz" # all alphabets
    letter_dict = {}
    sentence = sentence.lower() # don't need to distinguish between large and small le

    for i in letter: # add
        letter_dict[i] = 0
        for j in sentence:
            if i == j:
                letter_dict[i] += 1

    print(letter_dict) # print

    for i in letter_dict:
        if letter_dict[i] != 0:
            print("%s:%d" %(i, letter_dict[i])) # print for instrctions
```

```
    return letter_dict

char_count = letter_count("Four score and seven years ago our fathers brought forth on
lst = [char_count[i] for i in char_count] # add counting numbers
lst.sort() # sort it for finding third most frequent number
reverse_char_count = {i:j for j,i in char_count.items()} # make the new dictionary rev
print("Third most frequent letter: '%s' letter used %d times." %(reverse_char_count[ls
```

```
Name: Sehyoun Jang
sej324@lehigh.edu
Assignment ENGR10-S23-PY 7-3, Problem: 1
GROUP 3
------------------------------------------------------------------
{'a': 13, 'b': 2, 'c': 6, 'd': 7, 'e': 18, 'f': 3, 'g': 2, 'h': 6, 'i': 9, 'j': 0,
'k': 0, 'l': 4, 'm': 1, 'n': 14, 'o': 14, 'p': 2, 'q': 1, 'r': 11, 's': 6, 't': 15,
'u': 4, 'v': 2, 'w': 1, 'x': 0, 'y': 2, 'z': 0}
a:13
b:2
c:6
d:7
e:18
f:3
g:2
h:6
i:9
l:4
m:1
n:14
o:14
p:2
q:1
r:11
s:6
t:15
u:4
v:2
w:1
y:2
Third most frequent letter: 'o' letter used 14 times.
```

2(M). Use Python to determine and print out which, if any, letter in the alphabet does not appear in char_count dictionary created in Ex. 02.

In [47]:
```
Sehyoun_Jang(2)

print("Doest not appear letter in sentence: ")
for i in char_count:
    if char_count[i] == 0: # find the letter which does not appear in the sentence
        print(i, end = " ")
```

```
Name: Sehyoun Jang
sej324@lehigh.edu
Assignment ENGR10-S23-PY 7-3, Problem: 2
GROUP 3
------------------------------------------------------------
Doest not appear letter in sentence:
j k x z
```

3(E). Create my_struct dictionary which contains the following data:
{"Set 1":{"letters":["a","b","c"],"numbers":["1","2","3"]},"Set 2":[{"a":[10,20,30],"b":[100,101,102]},20,30]}

Using bracket/index notation, print:

- the character "b" in "letters" from my_struct
- the value 101 from my_struct

In [48]:
```
Sehyoun_Jang(3)
my_struct = {"Set 1":{"letters":["a","b","c"],"numbers":["1","2","3"]},"Set 2":[{"a":[

print(my_struct["Set 1"]["letters"][1])
print(my_struct["Set 2"][0]["b"][1])
```

```
Name: Sehyoun Jang
sej324@lehigh.edu
Assignment ENGR10-S23-PY 7-3, Problem: 3
GROUP 3
------------------------------------------------------------
b
101
```

4(M). Use my_struct dictionary created in Ex. 03, write only dictionary value of the key "Set 1" to a text file called my_dict.txt. Each line in my_dict.txt should consist of a key followed by the corresponding value, separated by a colon (:).

Read in my_dict.txt and store the data into a dictionary called my_struct2. Print my_struct2 and confirm that it is the same as the one stored inside my_struct.

In [49]:
```
Sehyoun_Jang(4)

with open("my_dict.txt", "w") as file:
    for key, value in my_struct["Set 1"].items():
        file.write(f"{key}:{value}\n")

my_struct2 = {}

alphabets = "abcdefghijklmnopqrstuvw"
numbers = "1234567890"

with open("my_dict.txt", "r") as file2:
    for line in file2:
        a, b = line.strip().split(":")
        temp = []
        for i in b:
            if i in alphabets or i in numbers: # for getting the elements from text_li
```

```
                temp.append(i)
        my_struct2[a] = temp

print("Check")
print(my_struct["Set 1"])
print(my_struct2)
```

```
Name: Sehyoun Jang
sej324@lehigh.edu
Assignment ENGR10-S23-PY 7-3, Problem: 4
GROUP 3
------------------------------------------------------------------
Check
{'letters': ['a', 'b', 'c'], 'numbers': ['1', '2', '3']}
{'letters': ['a', 'b', 'c'], 'numbers': ['1', '2', '3']}
```

5(M). Using list addition, combine the two lists [1,2,5,2,5,1,7,4] and [3,2,6,9,7,1,7,8] into comb_list. Then:

- **Without the use of a For loop or While loop, including those in list comprehension**, find and print out the unique elements within comb_list as a list.
- Print out the elements, if any, that occur more than once in comb_list as a list.
- Print out the elements, if any, that occur only once in comb_list as a list.

In [50]:
```
Sehyoun_Jang(5)

a = [1,2,5,2,5,1,7,4]
b = [3,2,6,9,7,1,7,8]

comb_list = a + b
comb_list.sort()
lst_1 = list(set(comb_list)) # unique elements
lst_2 = list(set((filter(lambda x: comb_list.count(x) > 1, comb_list)))) # more than c
lst_3 = list(filter(lambda x: comb_list.count(x) == 1, comb_list)) # only once occure


print(lst_1)
print(lst_2)
print(lst_3)
```

```
Name: Sehyoun Jang
sej324@lehigh.edu
Assignment ENGR10-S23-PY 7-3, Problem: 5
GROUP 3
------------------------------------------------------------------
[1, 2, 3, 4, 5, 6, 7, 8, 9]
[1, 2, 5, 7]
[3, 4, 6, 8, 9]
```

6(M). Define a function with one parameter. The inputted parameter should be a list and the function should return two lists: one containing elements that are duplicated, that is, the elements that occur more than once, one is a sorted list containing the unique elements within the inputted list. Call your function six times, using each of the following as the inputted list, and print out the returned result:

- The list comb_list created in Ex. 05
- A list with no duplicates
- A list with just one duplicate
- A list with two duplicates and several non-duplicates
- A list with only duplicates
- A list where all the elements are the same

In [51]:
```
Sehyoun_Jang(6)

def func1(lst):

    lst_1 = list(set(lst)) # unique elements
    lst_1.sort()
    lst_2 = list(set((filter(lambda x: lst.count(x) > 1, lst)))) # more than once occu
    lst_2.sort()

    return lst_1, lst_2
#1
print(func1(comb_list))
#2
temp1 = [1,4,5,6,7] # no duplication
print(func1(temp1))
#3
temp2 = [1,1] # one duplication
print(func1(temp2))
#4
temp3 = [1,2,2,3,3,5,6,7,8] # two duplication and several non-duplication
print(func1(temp3))
#5
temp4 = [1,1,4,4,7,7,8,8] # only duplication
print(func1(temp4))
#6
temp5 = [1,1,1,1,1,1,1,1,1] # all elements are the same
print(func1(temp5))
```

```
Name: Sehyoun Jang
sej324@lehigh.edu
Assignment ENGR10-S23-PY 7-3, Problem: 6
GROUP 3
----------------------------------------------------------------
([1, 2, 3, 4, 5, 6, 7, 8, 9], [1, 2, 5, 7])
([1, 4, 5, 6, 7], [])
([1], [1])
([1, 2, 3, 5, 6, 7, 8], [2, 3])
([1, 4, 7, 8], [1, 4, 7, 8])
([1], [1])
```

7(E). Import any necessary module, if it has not been imported. Create a dictionary called weights where the keys are the week and the values are the corresponding lists of weights, using the data below:

2019-12-15-2019-12-21:[151.6,152,151.8,151.6,150.8,152.2,151.2]

2019-12-21-2019-12-28:[153.2,152.4,150.8,151.6,154.4,153.8,152.2]

2019-12-29-2020-01-04:[152.8,152.4,153,154.8,153.8,151.2,152]

2020-01-05-2020-01-11:[150.8,150.2,149.8,149.8,150.4,150.6,1]
2020-01-12-2020-01-18:[151,150,150.2,152.2,152.4,152.4,153.2]
2020-01-19-2020-01-25:[153,152.4,151.2,151.2,151,151.2,151.2]
2020-01-26-2020-02-01:[152,150.8,151.2,150,150.8,152.4,151.2]

Print out weights as is.

In [52]:
```
Sehyoun_Jang(7)

import datetime

weights_date = [
    datetime.date(2019, 12, 15),
    datetime.date(2019, 12, 22),
    datetime.date(2019, 12, 29),
    datetime.date(2020, 1, 5),
    datetime.date(2020, 1, 12),
    datetime.date(2020, 1, 19),
    datetime.date(2020, 1, 26)
]
weights_data = [
    [151.6, 152, 151.8, 151.6, 150.8, 152.2, 151.2],
    [153.2, 152.4, 150.8, 151.6, 154.4, 153.8, 152.2],
    [152.8, 152.4, 153, 154.8, 153.8, 151.2, 152],
    [150.8, 150.2, 149.8, 149.8, 150.4, 150.6, 1],
    [151, 150, 150.2, 152.2, 152.4, 152.4, 153.2],
    [153, 152.4, 151.2, 151.2, 151, 151.2, 151.2],
    [152, 150.8, 151.2, 150, 150.8, 152.4, 151.2]
]

week_numbers = []
for i in weights_date:
    week_number = i.isocalendar()[1]
    week_numbers.append(week_number)

weights = {}
for i in range(len(week_numbers)):
    weights[week_numbers[i]] = weights_data[i]

print(weights)
print("print out")
for i in weights:
    print("%d week :" %i, end = " ")
    print(weights[i])
```

```
Name: Sehyoun Jang
sej324@lehigh.edu
Assignment ENGR10-S23-PY 7-3, Problem: 7
GROUP 3
-----------------------------------------------------------------
{50: [151.6, 152, 151.8, 151.6, 150.8, 152.2, 151.2], 51: [153.2, 152.4, 150.8, 151.
6, 154.4, 153.8, 152.2], 52: [152.8, 152.4, 153, 154.8, 153.8, 151.2, 152], 1: [150.
8, 150.2, 149.8, 149.8, 150.4, 150.6, 1], 2: [151, 150, 150.2, 152.2, 152.4, 152.4, 1
53.2], 3: [153, 152.4, 151.2, 151.2, 151, 151.2, 151.2], 4: [152, 150.8, 151.2, 150,
150.8, 152.4, 151.2]}
print out
50 week : [151.6, 152, 151.8, 151.6, 150.8, 152.2, 151.2]
51 week : [153.2, 152.4, 150.8, 151.6, 154.4, 153.8, 152.2]
52 week : [152.8, 152.4, 153, 154.8, 153.8, 151.2, 152]
1 week : [150.8, 150.2, 149.8, 149.8, 150.4, 150.6, 1]
2 week : [151, 150, 150.2, 152.2, 152.4, 152.4, 153.2]
3 week : [153, 152.4, 151.2, 151.2, 151, 151.2, 151.2]
4 week : [152, 150.8, 151.2, 150, 150.8, 152.4, 151.2]
```

8(E). Import any necessary module, if it has not been imported. Using the dictionary weights created in Ex. 09, calculate the average weight for each week, then determine and print out the highest and lowest average weight for the seven weeks. Plot all seven averages so you can visually see how they compare.

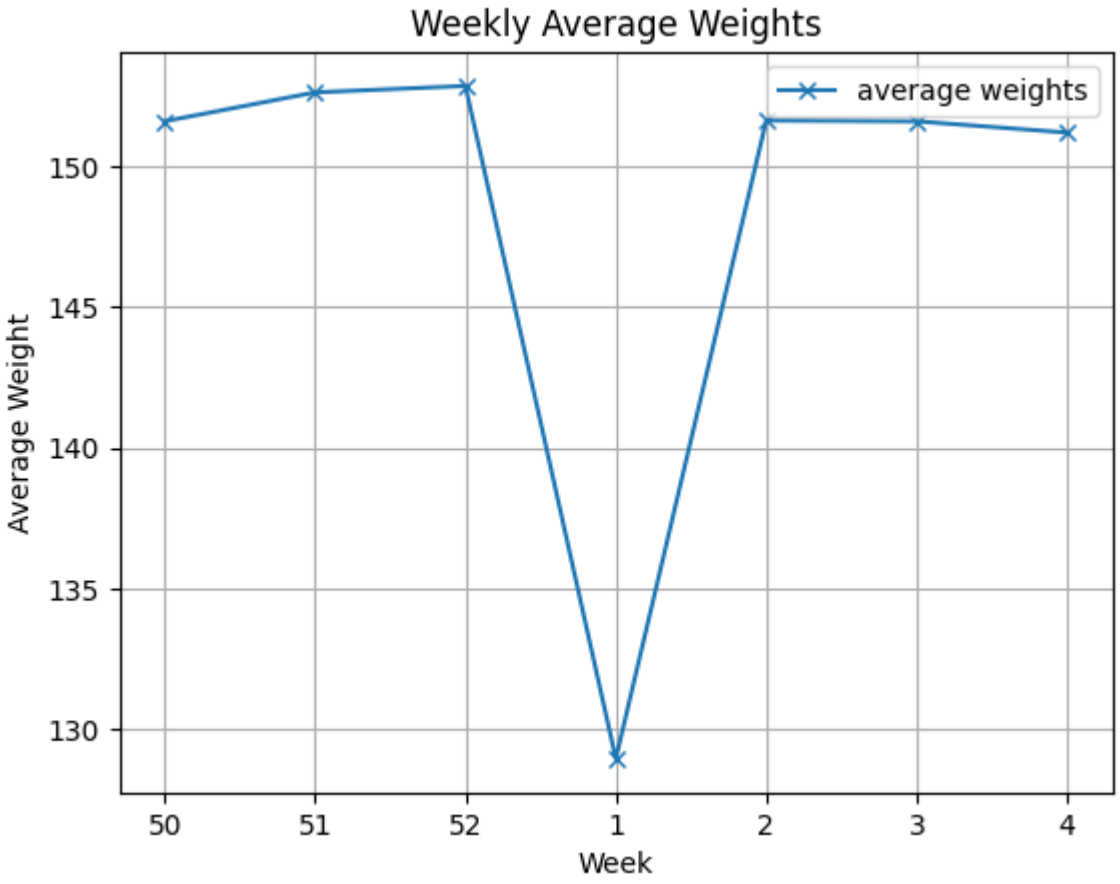
Then, on a separate graph, plot all the data from the weeks of the highest and lowest averages against their indices on a single chart so you can visually compare the two weeks. Be sure to use different colors and symbols for each week.

Remember to include details for an informative plot (such as legends, titles, axis labels,...).

In [53]:
```
Sehyoun_Jang(8)

import matplotlib.pyplot as plt

avg_lst = []
weights_keys = list(weights.keys())

for i in weights:
    avg = (sum(weights[i])/len(weights[i]))
    avg_lst.append(avg)
    highest_average_idx = avg_lst.index(max(avg_lst))
    lowest_average_idx = avg_lst.index(min(avg_lst))

print("highest average: %.2f in %d week" %(max(avg_lst), weights_keys[highest_average_
print("lowest average: %.2f in %d week" %(min(avg_lst), weights_keys[lowest_average_id


weights_keys_string = list(map(str, weights_keys))
plt.plot(weights_keys_string, avg_lst, marker='x', linestyle='-', label = 'average wei
plt.xlabel('Week')
plt.ylabel('Average Weight')
plt.title('Weekly Average Weights')
plt.grid()
plt.legend()
```

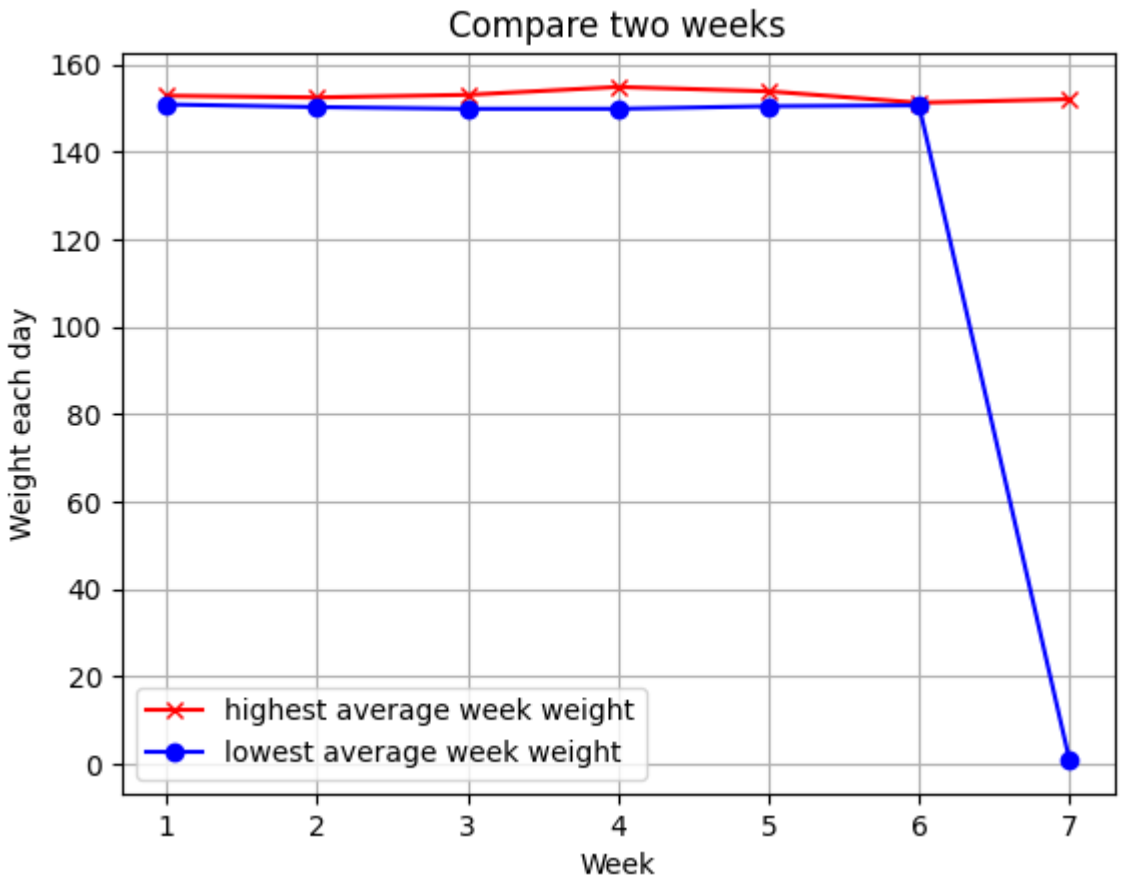
```
highest_week = weights[weights_keys[highest_average_idx]]
lowest_week = weights[weights_keys[lowest_average_idx]]
days = [1,2,3,4,5,6,7]
```

```
Name: Sehyoun Jang
sej324@lehigh.edu
Assignment ENGR10-S23-PY 7-3, Problem: 8
GROUP 3
------------------------------------------------------------------
highest average: 152.86 in 52 week
lowest average: 128.94 in 1 week
```



In [55]:
```
Sehyoun_Jang(8)
plt.plot(days, highest_week, color = 'red' ,marker = 'x', linestyle = '-', label = 'hi
plt.plot(days, lowest_week, color = 'blue', marker = 'o', linestyle = '-', label = 'lo
plt.xlabel('Week')
plt.ylabel('Weight each day')
plt.title('Compare two weeks')
plt.legend()
plt.grid()
```

```
Name: Sehyoun Jang
sej324@lehigh.edu
Assignment ENGR10-S23-PY 7-3, Problem: 8
GROUP 3
------------------------------------------------------------------
```



9(E). Write the dictionary weights created in Ex. 07 to weights_output.csv such that each week should be a row in the file, beginning with the week name and followed by the data. Once you have written the file, create a new dictionary called weights2 then read the contents of weights_output.csv into weights2. Print each element of weights and weights2 and confirm that the data is preserved correctly.

In [56]:
```
Sehyoun_Jang(9)
import csv

weights2 = {}
weights_keys = list(weights.keys())
weights_value = [weights[i] for i in weights_keys]
infor_weights = [[weights_keys[i]] + weights_value[i] for i in range(len(weights_keys)

f = open("weights_output.csv", 'w', newline = '')
fw = csv.writer(f)
for i in infor_weights:
    fw.writerow(i)
f.close()

f2 = open("weights_output.csv", 'r', newline = '')
fr = csv.reader(f2)
for i in fr:
    weights2[int(i[0])] = list(map(float, i[1:]))
f2.close()

print("original weights")
for i in weights:
```

```
    print("%d week : " %i, end = "")
    print(weights[i])
print("copied weights")
for j in weights2:
    print("%d week : " %j, end = "")
    print(weights2[j])
```

```
Name: Sehyoun Jang
sej324@lehigh.edu
Assignment ENGR10-S23-PY 7-3, Problem: 9
GROUP 3
-----------------------------------------------------------------
original weights
50 week : [151.6, 152, 151.8, 151.6, 150.8, 152.2, 151.2]
51 week : [153.2, 152.4, 150.8, 151.6, 154.4, 153.8, 152.2]
52 week : [152.8, 152.4, 153, 154.8, 153.8, 151.2, 152]
1 week : [150.8, 150.2, 149.8, 149.8, 150.4, 150.6, 1]
2 week : [151, 150, 150.2, 152.2, 152.4, 152.4, 153.2]
3 week : [153, 152.4, 151.2, 151.2, 151, 151.2, 151.2]
4 week : [152, 150.8, 151.2, 150, 150.8, 152.4, 151.2]
copied weights
50 week : [151.6, 152.0, 151.8, 151.6, 150.8, 152.2, 151.2]
51 week : [153.2, 152.4, 150.8, 151.6, 154.4, 153.8, 152.2]
52 week : [152.8, 152.4, 153.0, 154.8, 153.8, 151.2, 152.0]
1 week : [150.8, 150.2, 149.8, 149.8, 150.4, 150.6, 1.0]
2 week : [151.0, 150.0, 150.2, 152.2, 152.4, 152.4, 153.2]
3 week : [153.0, 152.4, 151.2, 151.2, 151.0, 151.2, 151.2]
4 week : [152.0, 150.8, 151.2, 150.0, 150.8, 152.4, 151.2]
```

10(H). Import any necessary module, if it has not been imported. Define a function with three parameters. Inside the function, create a dictionary final_dict where its keys are integers between first and second parameters. The corresponding value for each key should be a list of the prime factors of the number. In cases where a prime is repeated (such as 3*3 = 9) the prime should also be repeated in the list (e.g., the value for the key 14 should be [2, 7], the value for 28 should be [2, 2, 7]).

Once final_dict is fully generated, using random, print out a random number of keys and values, with each key and its corresponding value on a separate line, with a colon (:) separating the two. The number of key/value pair is determined by the third inputted parameter.

Call your function using 1, 100, and 10 as the three inputted parameters.

In [90]:
```
Sehyoun_Jang(10)
import random

def all_prime(number):
    count = 0
    lst = []
    for x in range(2, number + 1):
        count = 0
        for i in range (2, number + 1):
            if x % i == 0:
                count += 1
        if count == 1:
            lst.append(x)
```

```
    return lst

def prime_factors(number, prime_lst):
    lst = []
    for i in prime_lst:
        while True:
            if number % i == 0:
                lst.append(i)
                number = number / i
            else:
                break
    return lst

def func2(x, y, z):
    between_integer = [i for i in range (x, y+1)]
    prime_lst = all_prime(max(between_integer))

    final_dict = {}
    for i in range (x, y+1):
        final_dict[i] = prime_factors(i, prime_lst)

    for j in range (z):
        temp = random.randint(1, max(between_integer))
        print("%d number has prime numbers :" %temp, final_dict[temp])
    return

func2(1, 100, 10)
```

```
Name: Sehyoun Jang
sej324@lehigh.edu
Assignment ENGR10-S23-PY 7-3, Problem: 10
GROUP 3
----------------------------------------------------------------
78 number has prime numbers : [2, 3, 13]
79 number has prime numbers : [79]
60 number has prime numbers : [2, 2, 3, 5]
32 number has prime numbers : [2, 2, 2, 2, 2]
62 number has prime numbers : [2, 31]
63 number has prime numbers : [3, 3, 7]
25 number has prime numbers : [5, 5]
9 number has prime numbers : [3, 3]
40 number has prime numbers : [2, 2, 2, 5]
96 number has prime numbers : [2, 2, 2, 2, 2, 3]
```